

nanoGPT Revision Notes

Compact memory-sheet through model forward pass, optimizer setup, and generation loop

Scope: from imports and model building to AdamW + text generation. Training loop notes are not included yet.

Legend	B=batch, T=sequence length, C=embedding size, V=vocab size, nh=heads, hs=head size
Core flow	idx -> token embeddings + position embeddings -> blocks -> ln_f -> lm_head -> logits -> loss / sampling
Main idea	Attention mixes tokens; MLP transforms each token; residuals keep old info; final head scores vocab
Training vs generation	targets present -> cross_entropy loss; targets=None -> generate only logits

1) Python / PyTorch building blocks

These are the helper ideas that make the model code clean.

<code>from dataclasses import dataclass</code>	Standard Python decorator that turns a data-only class into a compact config container with auto-generated <code>__init__</code> .
<code>@dataclass</code> <code>class GPTConfig: ...</code>	Stores model hyperparameters in one object: <code>block_size</code> , <code>vocab_size</code> , <code>n_layer</code> , <code>n_head</code> , <code>n_embd</code> .
<code>class GPT(nn.Module):</code>	GPT inherits from PyTorch Module, so it gets parameter tracking, <code>.to()</code> , <code>.state_dict()</code> , <code>train/eval</code> , and <code>model(x) -> forward(x)</code> .
<code>self.transformer = nn.ModuleDict({...})</code>	A PyTorch-aware dictionary of submodules; PyTorch registers every module inside it.
<code>self.transformer.h = nn.ModuleList([Block(config) for _ in range(block_size)])</code>	A PyTorch-aware list of repeated blocks; each Block is a new object with its own weights.
<code>nn.Linear(in, out)</code>	Learnable affine layer: input features -> output features, applied on the last dimension.
<code>nn.Embedding(vocab_size, n_embd)</code>	Learnable lookup table: integer token IDs -> embedding vectors.
<code>nn.LayerNorm(n_embd)</code>	Normalizes each token over the last dimension only; keeps shape the same.
<code>nn.GELU(approximate="tanh")</code>	Smooth nonlinearity that breaks the "all linear layers collapse into one linear layer" problem.
<code>nn.Dropout(p)</code>	During training, randomly zeros some values to reduce overfitting; off in eval mode.
<code>assert condition</code>	Python safety check: if the condition is false, stop immediately with <code>AssertionError</code> and the message.
<code>f"...{x}..."</code>	Python f-string: inserts the value of <code>x</code> into the string.
<code>x.view(...)</code>	Reshapes the same tensor memory without changing values; requires compatible layout.
<code>x.transpose(dim1, dim2)</code>	Swaps two dimensions of a tensor; usually changes the view/layout, not the values.
<code>x.contiguous()</code>	Makes tensor memory contiguous again after transpose so <code>view()</code> can work safely.
<code>x[:, -n:]</code>	Keep all batches, last <code>n</code> tokens only.
<code>tensor.size(-1)</code>	Returns the size of the last dimension.

<code>tensor.dim()</code>	Returns the number of dimensions in the tensor.
<code>tensor.device / tensor.dtype / tensor.shape</code>	Tensor metadata: where it lives, what type it stores, and its shape.
<code>torch.no_grad()</code>	Inference mode: do not build a gradient graph; saves memory and speed.
<code>torch.arange(0, T, dtype=torch.long, device=idx.device)</code>	Create token-position IDs 0..T-1 on the same device as idx, using integer type for embedding lookup.
<code>torch.topk(x, k)</code>	Keep the k largest values and their indices.
<code>torch.multinomial(probs, 1)</code>	Sample one index according to the probability distribution.
<code>torch.cat((a, b), dim=1)</code>	Concatenate tensors along the token dimension to append the new token.
<code>torch.optim.AdamW(...)</code>	Optimizer that updates parameters using Adam + decoupled weight decay.
<code>inspect.signature(fn).parameters</code>	Python introspection: inspect a function's argument names without running it.

2) Attention block

The token-to-token communication engine.

<code>self.c_attn = nn.Linear(C, 3*C)</code>	One projection produces Query, Key, and Value together for efficiency.
<code>q, k, v = qkv.split(C, dim=2)</code>	Split the big QKV tensor into three tensors along the feature dimension.
<code>k = k.view(B, T, nh, C // nh).transpose(1, 2)</code>	Split embedding features into heads, then move head dimension before time: (B, nh, T, hs).
<code>F.scaled_dot_product_attention(q, k, v, is_causal=True)</code>	PyTorch's fast attention kernel: compute scores, scale, mask future tokens, softmax, and mix values.
<code>scores = q @ k.transpose(-2, -1)</code>	Dot products between queries and keys measure token relevance.
<code>scores = scores / sqrt(hs)</code>	Scale scores so softmax does not become too peaky when head size grows.
<code>causal mask / -inf before softmax</code>	Future tokens are forbidden; masked scores become -inf so softmax gives them probability 0.
<code>weights = softmax(scores)</code>	Convert raw scores into probabilities across the time dimension.
<code>output = weights @ v</code>	Weighted sum of value vectors: this is the actual information gathering step.
<code>y = y.transpose(1, 2).contiguous().view(B, T, C)</code>	Reassemble head outputs back into one embedding per token.

<code>self.c_proj = nn.Linear(C, C)</code>	Final projection mixes information across heads before returning to the block.
<code>NANOGPPT_SCALE_INIT</code> on <code>c_proj</code>	Tag used by custom init to shrink residual-branch weights for stability.

3) MLP and Transformer Block

MLP = per-token thinking. Block = attention + MLP with residuals.

<code>self.c_fc = nn.Linear(C, 4*C)</code>	Expand each token to a wider hidden space so it can learn richer features.
<code>self.gelu = nn.GELU(approximate="tanh")</code>	Add nonlinearity; without it, stacked Linear layers collapse into one linear transform.
<code>self.c_proj = nn.Linear(4*C, C)</code>	Compress the expanded hidden state back to the model width.
<code>x = self.c_fc(x) -> GELU -> c_proj</code>	MLP pipeline: expand, transform nonlinearly, compress.
<code>x = x + self.attn(self.ln_1(x))</code>	Pre-LN residual block: normalize, attend across tokens, then add the result back to the original x.
<code>x = x + self.mlp(self.ln_2(x))</code>	Second residual path: normalize, process each token independently, then add back.
residual connection: <code>x + ...</code>	Keeps old information alive and helps gradients flow through deep stacks.
Block output shape stays <code>(B, T, C)</code>	Every block changes meaning, not the shape, so blocks can stack cleanly.

4) GPT assembly: embeddings -> blocks -> logits

This is how the complete decoder-only Transformer is built.

<code>wte = nn.Embedding(vocab_size, n_embd)</code>	Token embedding lookup: token ID -> learned vector.
<code>wpe = nn.Embedding(block_size, n_embd)</code>	Position embedding lookup: position index -> learned vector.
<code>tok_emb = self.transformer.wte(idx)</code>	Convert token IDs (B,T) into token vectors (B,T,C).
<code>pos = torch.arange(0, T, dtype=torch.long, device=idx.device)</code>	Create position IDs 0..T-1 on the same device as the input tokens.
<code>pos_emb = self.transformer.wpe(pos)</code>	Convert position IDs into learned position vectors.
<code>x = tok_emb + pos_emb</code>	Add token identity and position information; shapes match by broadcasting.
<code>x = self.transformer.drop(x)</code>	Dropout for regularization during training only.
<code>for block in self.transformer.h: x = block(x)</code>	Send x through every Transformer block in order.

<code>x = self.transformer.ln_f(x)</code>	Final LayerNorm before prediction; stabilizes the last hidden state.
<code>self.lm_head = nn.Linear(n_embd, vocab_size, bias=False)</code>	Map each token representation to one score per vocabulary token.
<code>logits = self.lm_head(x)</code>	Raw vocabulary scores for every token position: (B, T, V).
logits are raw scores, not probabilities	Logits can be negative or >1; softmax turns them into probabilities later.
weight tying: <code>self.transformer.wte.weight = self.lm_head.weight</code>	Input embedding matrix and output classifier share weights to reduce parameters and help consistency.

5) forward() and loss

This is the full prediction path used in both training and generation.

<code>def forward(self, idx, targets=None):</code>	One method handles both training (targets given) and generation (targets absent).
<code>B, T = idx.size()</code>	Read batch size and sequence length from the input token tensor.
<code>assert T <= block_size</code>	Fail fast if the prompt is longer than the model context window.
<code>if targets is not None:</code>	Only compute loss during training; generation skips it.
<code>loss = None</code>	Default value so the same forward method can return logits only when needed.
<code>logits.view(-1, logits.size(-1))</code>	Flatten B and T into one prediction axis, keep vocab dimension intact.
<code>targets.view(-1)</code>	Flatten the correct token IDs into one target vector aligned with the flattened logits.
<code>F.cross_entropy(flat_logits, flat_targets)</code>	Compare raw logits with correct IDs and produce one scalar loss.
<code>return logits, loss</code>	Always return logits; return loss only when targets exist.
<code>logits.shape = (B, T, V)</code>	One vocab score vector per token position.
<code>targets.shape = (B, T)</code>	One correct token ID per position.
<code>logits[:, -1, :]</code>	Keep only the last token's prediction when generating the next token.

6) Optimizer setup

This part decides how weights actually get updated during training.

<code>self.named_parameters()</code>	PyTorch returns every parameter with its full name.
--------------------------------------	---

<code>{pn: p for pn, p in self.named_parameters()}</code>	Dictionary comprehension: build name -> parameter lookup.
<code>if p.requires_grad</code>	Keep only trainable parameters; frozen tensors are skipped.
<code>p.dim() >= 2</code>	Select matrix-like parameters: Linear weights and embedding tables.
<code>p.dim() < 2</code>	Select 1D parameters: biases and LayerNorm params.
<code>decay_params / no_decay_params</code>	Split params into weight-decay and no-decay groups.
<code>weight_decay</code>	Small penalty that discourages overly large weights and helps generalization.
<code>optim_groups = [{...}, {...}]</code>	PyTorch optimizer groups let different parameter sets use different hyperparameters.
<code>'fused' in inspect.signature(torch.optim.AdamW).parameters</code>	Feature-detect whether this PyTorch version supports fused AdamW.
<code>use_fused = fused_available and device == "cuda"</code>	Only enable fused AdamW when supported and running on CUDA.
<code>torch.optim.AdamW(..., lr=learning_rate, betas=(0.9, 0.95), eps=1e-8, optimizer_fused=use_fused)</code>	Create the optimizer. If use_fused is set, betas = memory of past gradients, eps = numerical safety.
<code>optimizer.step()</code>	Actually update parameters using the gradients; backprop alone does not change weights.

7) generate(): text sampling loop

This is the autoregressive loop that makes GPT write token by token.

<code>@torch.no_grad()</code>	Inference only: do not build a gradient graph; saves memory and speed.
<code>def generate(self, idx, max_new_tokens, temperature=1.0, top_k=None):</code>	Start from a prompt, generate N new tokens, optionally control randomness and top-k filtering.
<code>for _ in range(max_new_tokens):</code>	Repeat the generation step once per new token.
<code>idx_cond = idx if idx.size(1) <= block_size else idx[:, -block_size:]</code>	Keep the whole prompt if short enough; otherwise keep only the last block_size tokens.
<code>logits, _ = self(idx_cond)</code>	Run the model on the current context; ignore loss during generation.
<code>logits = logits[:, -1, :]</code>	Use only the last position because it predicts the next token after the full context.
<code>logits = logits / temperature</code>	Lower temperature sharpens choices; higher temperature makes output more random.

<code>if top_k is not None: keep top_k logits, set rest to -inf</code>	Remove very unlikely tokens before sampling.
<code>probs = F.softmax(logits, dim=-1)</code>	Turn scores into a probability distribution over the vocabulary.
<code>idx_next = torch.multinomial(probs, num_samples=1)</code>	Sample one next token according to the probability distribution, not necessarily the argmax.
<code>idx = torch.cat((idx, idx_next), dim=1)</code>	Append the sampled token to the sequence and repeat.
<code>temperature < 1 / > 1</code>	Below 1 = more deterministic; above 1 = more diverse.
<code>top_k filtering</code>	Keep only the K most likely tokens so sampling stays coherent.
<code>autoregressive generation</code>	Predict one token, append it, feed it back, and repeat.

End of current scope: covered from imports through optimizer + generation. Training loop notes can be added later.